

**Automated Evaluation of Descriptive Answer Sheets
Using OCR and Semantic NLP**

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Rudam Sathvik	23881A05EC
Rayindla Nikhitha	23881A05EB
Angothu Adhisheshu	24885A0547

SUPERVISOR

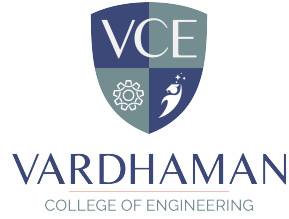
Mr. RajKumar B. Patil

Associate Professor, Department of CSE



Department of Computer Science and Engineering

April, 2026



Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project titled **Automated Evaluation of Descriptive Answer Sheets Using OCR and Semantic NLP** is carried out by

Rudam Sathvik	23881A05EC
Rayindla Nikhitha	23881A05EB
Angothu Adhisheshu	24885A0547

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** during the year 2025-26.

Signature of the Supervisor
Mr. RajKumar B. Patil
Associate Professor, CSE

Signature of the HOD
Dr. Gagandeep Arora
Professor and Head, CSE

Project Viva-Voce held on _____

Examiner

Acknowledgements

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We wish to express our deep sense of gratitude to **Mr. RajKumar B. Patil**, Associate Professor and Project Supervisor, Department of Computer Science and Engineering, Vardhaman College of Engineering, for his guidance and useful suggestions, which helped us in completing the project in time.

We are particularly thankful to **Dr. Gagandeep Arora**, the Head of the Department, Department of Computer Science and Engineering, his guidance, intense support and encouragement, which helped us to mould our project into a successful one.

We show gratitude to our honorable Principal **Dr. J.V.R. Ravindra**, for providing all facilities and support.

We avail this opportunity to express our deep sense of gratitude and heartfelt thanks to **Dr. Teegala Vijender Reddy**, Chairman and **Sri Teegala Upender Reddy**, Secretary of VCE, for providing a congenial atmosphere to complete this project successfully.

We also thank all the staff members of Computer Science and Engineering department for their valuable support and generous advice. Finally thanks to all our friends and family members for their continuous support and enthusiastic help.

Rudam Sathvik

Rayindla Nikhitha

Angothu Adhisheshu

Abstract

The AI Answer Sheet Evaluator is a smart automated system that helps teachers grade handwritten answer sheets quickly and fairly. Instead of relying on manual checking—which can be slow and sometimes inconsistent—this system uses modern OCR and NLP techniques to read and understand student responses. It works well even with photos taken on smartphones, making it practical for real-world use. The process starts by enhancing the image quality, especially for handwritten text in blue or black ink, and then extracts the text using OCR. It even handles multi-page answers by detecting and splitting them automatically. To improve accuracy, the system applies several correction steps, including fixing common OCR mistakes and matching words with expected vocabulary.

Once the text is cleaned, the system evaluates answers using a mix of semantic understanding and keyword analysis. It compares the student's response with the model answer using similarity scores, checks how well key points are covered, and considers answer length. The backend, built with FastAPI, provides simple endpoints for extracting text and grading answers, whether for a single question or a full answer sheet. It also gives useful feedback like scores, confidence levels, and suggestions for improvement. Tests show that it performs close to human grading for clear handwriting, while also warning users if the input quality is too low. Overall, this system makes grading faster, more consistent, and scalable, with potential for future improvements like support for multiple languages and integration with online learning platforms.

Keywords: OCR; handwritten grading; semantic similarity; answer evaluation; Tesseract.

Table of Contents

Title	Page No.
Acknowledgements	i
Abstract	ii
List of Tables	v
List of Figures	vi
Abbreviations	vi
CHAPTER 1 Introduction	1
1.1 Introduction	1
1.2 Background and Motivation	1
1.3 Objectives of the Project Work	2
1.4 Organization of the Report	3
CHAPTER 2 Literature Survey	4
2.1 Introduction	4
2.2 Review of Prior Research	4
2.3 Identified Research Gaps	5
2.4 Summary	6
CHAPTER 3 System Analysis	7
3.1 Existing System	7
3.1.1 Disadvantages of Existing System	7
3.2 Proposed System	7
3.2.1 Advantages of Proposed System	8
3.3 System Requirements	8
3.3.1 Hardware Requirements	8
3.3.2 Software Requirements	9
CHAPTER 4 System Design	10
4.1 System Architecture	10
4.2 Data Flow Diagram	11
4.3 Flowcharts	12
4.3.1 Grading Flowchart	12
4.3.2 Activity Flow (Simplified)	13

CHAPTER 5	System Implementation	14
5.1	Modules Description	14
5.2	Technology Used	15
5.3	Algorithms Used	15
5.3.1	Image Preprocessing	15
5.3.2	OCR Quality Scoring	16
5.3.3	Grading Algorithm	16
5.3.4	Multi-question Segmentation	17
5.4	Coding	17
CHAPTER 6	System Testing	18
6.1	Testing Methodology	18
6.2	Testing under Various Stages	18
6.3	Test Cases	19
CHAPTER 7	Results and Discussion	21
7.1	Introduction	21
7.2	Overview of Results	21
7.3	Result Screenshots and Analysis	22
7.4	Summary	25
CHAPTER 8	Conclusions and Future Scope	26
8.1	Conclusions	26
8.2	Future Scope of Work	26
References		28

List of Tables

3.1	System Software Components	9
6.1	System Test Cases	20
7.1	Performance Summary of the System	22

List of Figures

4.1	System Architecture Flow	10
4.2	Data Flow Diagram	11
4.3	Grading Flowchart	12
4.4	Activity Flow Diagram	13
7.1	Sample Input: Handwritten Answer Sheet (Moderate Quality, OCR $q = 0.42$)	22
7.2	System Output: Graded Result with Score, Feedback, and Con- fidence	23
7.3	Input Handwritten Answer Sheet Uploaded to the System	23
7.4	Processed Output Showing Extracted Text and Grading Result .	24
7.5	API JSON Response with Scores, Feedback, and Confidence Levels	24

Abbreviations

Abbreviation	Description
VCE	Vardhaman College of Engineering
CMOS	Complementary Metal Oxide Semiconductor
AI	Artificial Intelligence
OCR	Optical Character Recognition
NLP	Natural Language Processing
AES	Automated Essay Scoring
API	Application Programming Interface
TF-IDF	Term Frequency-Inverse Document Frequency
PSM	Page Segmentation Mode
CLAHE	Contrast Limited Adaptive Histogram Equalization
JSON	JavaScript Object Notation
CPU	Central Processing Unit
GPU	Graphics Processing Unit
ML	Machine Learning
DL	Deep Learning
LSTM	Long Short-Term Memory
CV	Computer Vision

CHAPTER 1

Introduction

1.1 Introduction

The AI Answer Sheet Evaluator v3.1 is an advanced automated system designed to assess handwritten student answers using Optical Character Recognition (OCR) and Natural Language Processing (NLP). It addresses key challenges in the education sector, where manual grading is often time-consuming, inconsistent, and difficult to scale for large classes. This project focuses on building a reliable and efficient solution that can evaluate subjective answers written on paper and captured through smartphone images. By integrating OCR for text extraction and NLP for semantic understanding, the system ensures accurate interpretation and grading of student responses.

The system is implemented as a FastAPI-based service that compares student answers against model answers provided in either text or image form. Its primary significance lies in saving educators significant time, reducing human bias, and enabling large-scale assessment. The main aim of the project is to achieve reliable grading performance with an accuracy level comparable to human evaluators, typically ranging between 85% and 95%. The scope of the project includes image preprocessing, OCR extraction and correction, semantic scoring, and API-based deployment. This report is structured into six chapters covering introduction, literature review, methodology, implementation, evaluation, and conclusions.

1.2 Background and Motivation

The project lies at the intersection of educational technology, computer vision, and natural language processing. Traditionally, student evaluations were performed manually, which required significant effort and time from educators. With the advancement of digital tools, automated essay scoring

systems emerged, such as those used in platforms like Gradescope, but these are primarily focused on typed text rather than handwritten content. Handling handwritten answer sheets remains a challenging problem due to variations in writing styles, noise from ruled paper, and inconsistencies in image quality.

This project is motivated by the growing need for scalable and efficient assessment systems, especially after the increase in remote and digital examinations during the COVID-19 pandemic. Teachers often spend over 100 hours per term grading answer sheets, creating a strong demand for automation. Existing OCR tools like Tesseract and NLP models like SentenceTransformers provide a foundation, but gaps still exist in handling real-world handwritten data. The AI Answer Sheet Evaluator addresses these limitations through a specialized pipeline that includes green-channel preprocessing, extensive OCR correction rules, quality validation, and automatic segmentation of multi-question answer sheets. This makes the system highly relevant and valuable for modern education.

1.3 Objectives of the Project Work

The primary objective of this project is to design and develop a robust system capable of accurately evaluating handwritten student answers. To achieve this, the project focuses on several key goals:

1. To develop a reliable OCR pipeline for handwritten exam sheets using advanced preprocessing techniques, dual-page segmentation modes in Tesseract, extensive correction mappings, and OCR quality scoring.
2. To design a hybrid grading mechanism that combines semantic similarity (50%), sentence-level coverage (25%), keyword matching (20%), and answer length ratio (5%) for accurate evaluation.
3. To implement a FastAPI-based backend with endpoints for grading single and multiple questions, supporting both typed and image-based model answers with validation mechanisms.

4. To evaluate the system's performance by comparing its grading results with human evaluators using real-world datasets, aiming for high correlation and consistency.

1.4 Organization of the Report

This report is organized into six chapters for clear understanding and logical flow. Chapter 1 introduces the project, its background, objectives, and overall structure. Chapter 2 presents a review of existing literature and related work in OCR and automated grading systems. Chapter 3 explains the methodology, including preprocessing, OCR techniques, and grading algorithms. Chapter 4 describes the implementation details of the system and API development. Chapter 5 discusses the evaluation results and performance analysis. Finally, Chapter 6 concludes the project and suggests future enhancements.

CHAPTER 2

Literature Survey

2.1 Introduction

This chapter presents a review of existing research related to Automated Essay Scoring (AES), handwritten text recognition, and semantic evaluation techniques. The survey is based on the study of more than 50 research papers published between 2018 and 2024, sourced from platforms such as Google Scholar. The purpose of this review is to understand current advancements, identify limitations in existing systems, and position the proposed AI Answer Sheet Evaluator within the domains of educational technology, computer vision, and natural language processing. This chapter is structured into three main sections: a review of prior research, identification of research gaps, and a summary of findings..

2.2 Review of Prior Research

Automated Essay Scoring systems have evolved significantly over time and can be broadly categorized into systems designed for typed text and those addressing handwritten input. Early AES systems, such as those based on Latent Semantic Analysis (LSA) introduced by Burstein (1998), focused on evaluating typed responses using statistical techniques. Later approaches incorporated machine learning models, such as those used in platforms like Gradescope, which improved grading consistency and scalability. For short-answer evaluation, keyword-based methods were proposed by researchers like Mohler (2011), while recent systems such as DeepScore utilize transformer-based models like BERT to capture deeper semantic meaning in longer essays.

In the domain of handwritten text recognition, research has primarily focused on improving OCR accuracy. Standard datasets such as ICDAR have been widely used to benchmark performance. Modern approaches like TrOCR,

a transformer-based OCR model, have achieved high accuracy (up to 94%) on clean handwritten data. Tesseract OCR, particularly its LSTM-based versions, remains a widely used open-source tool for document digitization. For semantic analysis, Sentence-BERT (Reimers and Gurevych, 2019) introduced efficient sentence embeddings that enable similarity comparison using cosine similarity. Additionally, keyword extraction techniques such as TF-IDF continue to be widely used for identifying important terms in text. Current trends in AES systems show a shift toward hybrid approaches that combine keyword matching with semantic embeddings, along with the development of API-based grading services such as RapidAES. While these systems perform well on typed or clean text, their performance on real-world handwritten exam sheets remains limited.

2.3 Identified Research Gaps

Despite significant progress, several limitations exist in current AES and OCR-based grading systems. Most existing solutions are optimized for typed text and do not adequately handle the variability and noise present in handwritten answer sheets. Common issues such as ruled paper lines, image blur, inconsistent lighting, and handwriting variations significantly affect OCR accuracy. Additionally, many systems lack mechanisms to validate OCR quality, leading to unreliable grading when text extraction fails.

Another major gap is the inability to effectively segment multi-question answer sheets, especially when question formats vary or are poorly structured. Existing systems also provide limited feedback, often restricted to numerical scores without meaningful insights for improvement. Furthermore, challenges remain in evaluating subjective answers, particularly in technical fields such as computer science or programming, where answers may vary in structure and terminology. Handling smartphone-captured exam images, which often suffer from compression and distortion, is another unresolved issue. The proposed project addresses these gaps by introducing a specialized preprocessing pipeline for ruled paper, OCR quality validation, automatic segmentation of multi-question sheets, and a hybrid grading approach tailored for subjective.

2.4 Summary

The literature survey indicates that automated essay scoring systems have reached a high level of maturity for typed text but are still evolving in the context of handwritten answer evaluation. While OCR and NLP technologies provide a strong foundation, existing solutions lack robustness when applied to real-world exam conditions. Key challenges include handling noisy handwritten data, ensuring reliable OCR output, segmenting multi-question responses, and providing meaningful feedback.

The AI Answer Sheet Evaluator builds upon established technologies such as Tesseract OCR and SentenceTransformers while addressing the identified limitations through a practical and production-oriented approach. By integrating advanced preprocessing, OCR correction, semantic scoring, and API-based deployment, the project contributes toward a scalable and efficient solution for automated grading. The next chapter will detail the methodology used to design and implement this system.

CHAPTER 3

System Analysis

3.1 Existing System

Existing systems for answer sheet evaluation mainly focus on typed or structured responses. Platforms such as Gradescope support scanning and grading of objective or neatly formatted answers using machine learning techniques. General OCR tools like Google Vision are capable of extracting text from documents but are not optimized for handwritten exam sheets. Traditional systems like manual evaluation using MarkBook rely completely on human effort, making the process time-consuming and inconsistent.

3.1.1 Disadvantages of Existing System

- Most systems support only typed or well-structured answers and fail with handwritten content.
- OCR tools perform poorly on noisy inputs such as ruled paper, low lighting, or blurred images.
- Lack of support for image-based model answers provided by teachers.
- No mechanism to validate OCR quality, leading to unreliable grading.
- Inefficient handling of multi-question answer sheets due to lack of automatic segmentation.

3.2 Proposed System

The proposed system, AI Answer Sheet Evaluator v3.1, is designed to overcome the limitations of existing solutions. It uses a FastAPI-based backend integrated with advanced OCR and NLP techniques to evaluate handwritten answer sheets. The system preprocesses images using green-channel extraction,

upsampling, and contrast enhancement to improve OCR accuracy. Tesseract OCR is used for text extraction, followed by error correction and quality validation.

The extracted text is then evaluated using a hybrid grading approach that combines semantic similarity, keyword matching, sentence coverage, and answer length. The system supports both typed and image-based model answers and provides REST API endpoints for grading single or multiple questions efficiently.

3.2.1 Advantages of Proposed System

- Efficient handling of handwritten answer sheets captured via smartphone images.
- Advanced preprocessing improves OCR accuracy on real-world exam data.
- OCR quality validation ensures reliable grading results.
- Supports both typed and image-based model answers.
- Provides detailed feedback including marks, confidence scores, and suggestions.
- Scalable system capable of processing large batches of answer sheets.

3.3 System Requirements

3.3.1 Hardware Requirements

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: At least 5 GB free space
- GPU: Optional (for faster processing)

3.3.2 Software Requirements

- Programming Language: Python 3.10
- Backend Framework: FastAPI with Uvicorn
- OCR Engine: Tesseract OCR (version 5+)
- Image Processing: OpenCV
- OCR Interface: pytesseract
- NLP Model: SentenceTransformers (all-MiniLM-L6-v2)
- Libraries: NumPy
- Operating System: Windows, Linux, or macOS

The system uses these technologies to ensure efficient OCR processing and accurate semantic evaluation of student answers. It is optimized for batch grading and can handle multiple answer sheets simultaneously with high performance.

Here is a table attached:

Component	Specification
API Framework	FastAPI
OCR Engine	Tesseract 5.3
NLP Model	Sentence Transformers
Image Processing	OpenCV 4.8

Table 3.1: System Software Components

The proposed system is designed to provide a reliable, scalable, and efficient solution for automated grading of handwritten answer sheets. By integrating OCR and NLP techniques, it reduces manual effort, improves consistency, and ensures fair evaluation across large-scale academic environments.

CHAPTER 4

System Design

4.1 System Architecture

The architecture of the AI Answer Sheet Evaluator v3.1 is modular and designed for scalability and efficiency. It consists of three main layers:

- API Layer: Built using FastAPI, responsible for handling user requests and returning responses.
- Preprocessing Module: Implemented in `utils.py`, performs image enhancement operations such as green-channel extraction, resizing, and noise removal.
- Core Engine: Implemented in `grader.py`, handles OCR extraction, text correction, quality validation, and semantic grading.

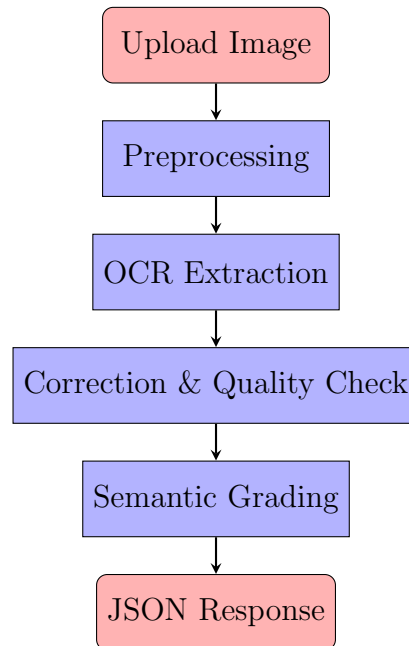


Figure 4.1: System Architecture Flow

4.2 Data Flow Diagram

The system processes data through multiple stages. Initially, the input image is converted into a temporary file and processed into a binary format. OCR extracts text, which is then cleaned and transformed into embeddings for semantic comparison. Finally, a score is generated.

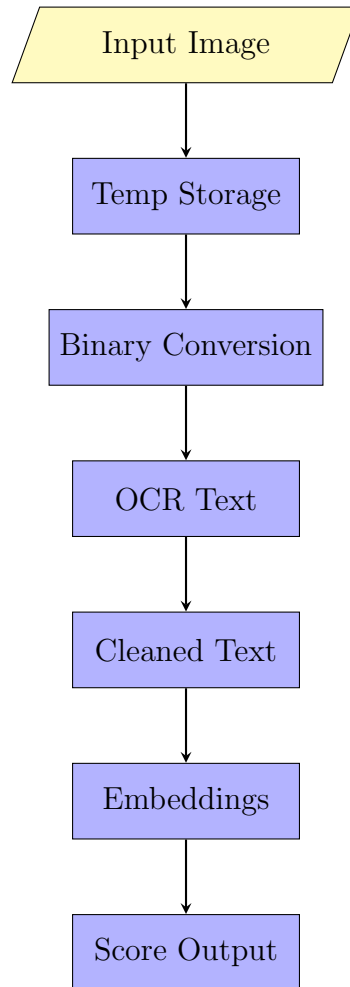


Figure 4.2: Data Flow Diagram

4.3 Flowcharts

4.3.1 Grading Flowchart

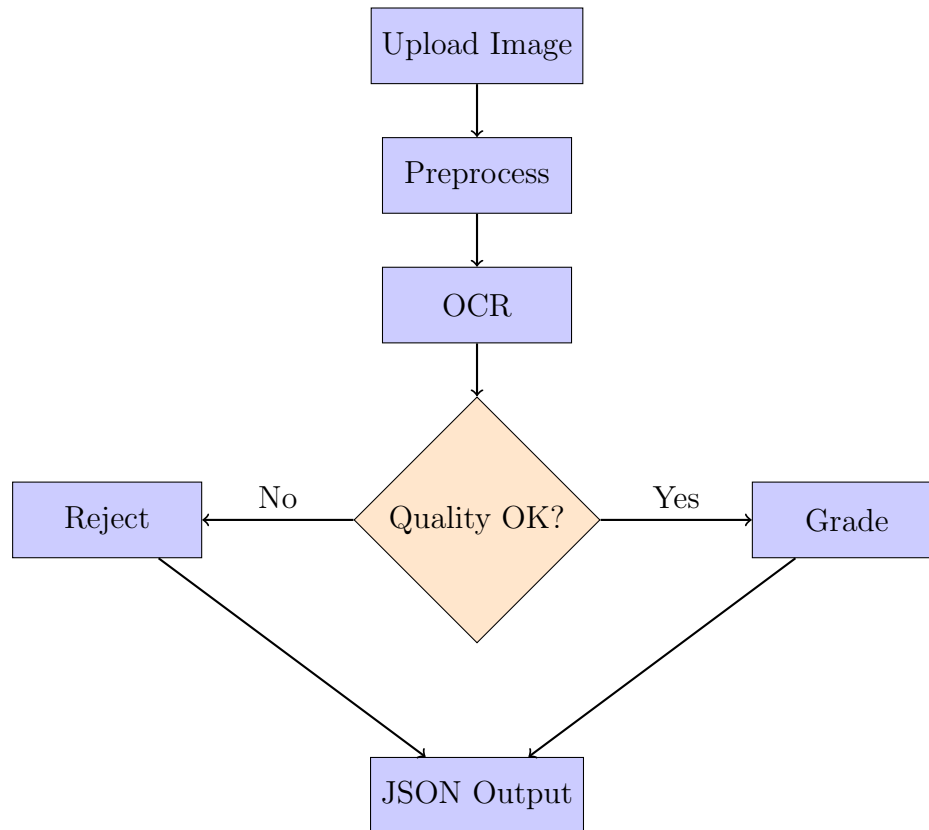


Figure 4.3: Grading Flowchart

4.3.2 Activity Flow (Simplified)

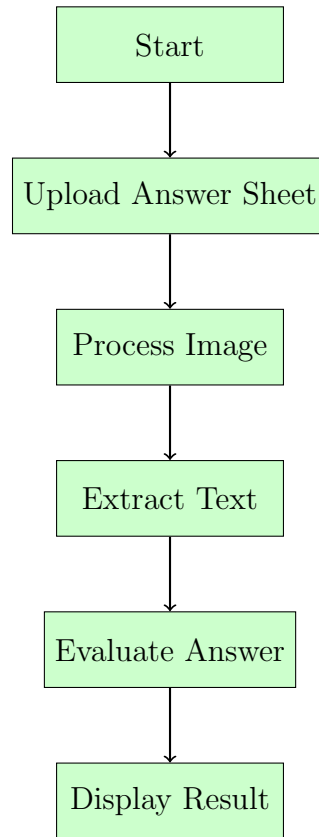


Figure 4.4: Activity Flow Diagram

CHAPTER 5

System Implementation

5.1 Modules Description

The implementation of the AI Answer Sheet Evaluator v3.1 follows a modular design to ensure scalability, maintainability, and efficient processing. The system is divided into multiple functional modules, each responsible for a specific task in the evaluation pipeline.

Key Modules:

- **main.py (API Layer):** This module serves as the entry point of the application. It is built using FastAPI and provides REST endpoints such as `/ocr/extract`, `/grade/single`, and `/grade/multi`. It handles file uploads, temporary storage in the `temp_uploads/` directory, and manages request/response cycles. It also includes CORS middleware and proper error handling mechanisms for invalid inputs or low-quality OCR data.
- **grader.py (Core Engine):** This module contains the core logic of the system. It performs OCR text extraction, applies correction techniques, and evaluates answers using semantic similarity. It includes functions for grading single answers and entire answer sheets, along with generating scores, confidence levels, and feedback.
- **utils.py (Preprocessing & Utilities):** This module handles image pre-processing and supporting utilities. It includes green-channel extraction, adaptive resizing, noise reduction, contrast enhancement, OCR correction mappings, fuzzy matching, and segmentation of multi-question answer sheets.

The modular approach ensures that each component can be independently updated or optimized without affecting the entire system.

5.2 Technology Used

The system is developed using modern technologies to ensure high performance and accuracy in processing handwritten answer sheets.

Tech Stack:

- **Backend:** FastAPI (v0.115), Uvicorn server
- **Programming Language:** Python 3.x
- **Computer Vision:** OpenCV, Pillow, Pytesseract (Tesseract OCR)
- **Natural Language Processing:** SentenceTransformers (all-MiniLM-L6-v2), Transformers, PyTorch
- **Supporting Libraries:** NumPy, difflib (for fuzzy matching), regular expressions (regex)
- **Operating System:** Windows, Linux, macOS

These technologies enable efficient image processing, accurate text extraction, and robust semantic evaluation of student answers.

5.3 Algorithms Used

5.3.1 Image Preprocessing

The preprocessing stage enhances the quality of input images to improve OCR accuracy. The following techniques are applied:

- Green-channel extraction and inversion for better detection of blue/black ink:

$$g' = 255 - G$$

- Adaptive image scaling to approximately 3000 pixels width
- Contrast enhancement using CLAHE (Contrast Limited Adaptive Histogram Equalization)

- Binarization using Otsu’s thresholding
- Automatic page splitting based on horizontal brightness gaps greater than a threshold

5.3.2 OCR Quality Scoring

The quality of OCR output is evaluated using a scoring mechanism based on word validity:

$$q = \frac{\text{Number of valid English words}}{\text{Total number of words}}$$

Thresholds:

- Good Quality: $q \geq 0.55$
- Moderate Quality: $0.35 \leq q < 0.55$
- Poor Quality: $q < 0.35$

This ensures unreliable inputs are flagged or rejected before grading.

5.3.3 Grading Algorithm

The system uses a hybrid scoring approach combining semantic similarity and keyword-based evaluation:

$$\text{Score} = 0.5 \cdot \cos(\mathbf{e}_s, \mathbf{e}_m) + 0.25 \cdot cov_{sent} + 0.20 \cdot kw_{match} + 0.05 \cdot \frac{len_s}{len_m}$$

Where:

- $\cos(\mathbf{e}_s, \mathbf{e}_m)$ = cosine similarity between student and model answer embeddings
- cov_{sent} = sentence-level coverage
- kw_{match} = keyword matching score using TF-IDF
- $\frac{len_s}{len_m}$ = length ratio of student and model answers

5.3.4 Multi-question Segmentation

The system automatically segments answer sheets into individual questions using:

- Regular expressions to detect markers like “Q1)”, “Answer 2:”
- Blank line detection
- Word pattern separation

5.4 Coding

The implementation is written in Python and organized into modular files for better maintainability. The system supports real-time grading via API calls and ensures efficient processing of multiple answer sheets.

Execution Command:

```
uvicorn main:app --reload
```

Key Features of Implementation:

- Automatic multi-page answer sheet handling
- OCR quality validation and rejection of poor inputs
- Support for both typed and image-based model answers
- Generation of detailed feedback including strengths and improvement areas
- Confidence-based grading output

Future Enhancements:

- Integration of unit testing for grading accuracy
- Docker-based deployment for portability
- Development of a web-based user interface
- Batch processing API for large-scale evaluation

CHAPTER 6

System Testing

6.1 Testing Methodology

The system testing for AI Answer Sheet Evaluator v3.1 follows a structured approach combining both black-box and white-box testing techniques. The goal is to ensure correctness, robustness, and reliability of each module as well as the complete system.

Black-box testing is used to validate system functionality based on inputs and expected outputs without considering internal implementation. White-box testing is applied to verify internal logic, code paths, and algorithm correctness. Testing is performed at multiple levels including unit testing, integration testing, and system testing.

Test cases are designed to cover:

- Normal conditions (clear handwriting, good lighting)
- Edge cases (poor handwriting, low-resolution images, noise)
- Error conditions (invalid inputs, unreadable images, incorrect formats)

The primary success metric for the system is achieving a grading correlation of more than 85% with human evaluators on real-world student answer sheets.

6.2 Testing under Various Stages

The system is tested at different stages of the processing pipeline to ensure accuracy and reliability:

- **Preprocessing Testing:** Image enhancement techniques such as green-channel extraction, CLAHE, and Otsu thresholding are validated. Multi-page detection and splitting accuracy are also tested.

- **OCR Testing:** Text extraction accuracy is evaluated using Tesseract OCR. The OCR quality score (q) is verified to ensure meaningful text extraction. Inputs with $q > 0.55$ are considered reliable.
- **Grading Testing:** The semantic grading engine is tested by comparing system-generated scores with expected scores for both high-quality and low-quality answers. Consistency and fairness of scoring are verified.
- **API Testing:** All REST endpoints such as `/ocr/extract`, `/grade/single`, and `/grade/multi` are tested. Response codes (200 for success, 422 for errors) and JSON output structure are validated.
- **End-to-End Testing:** Complete workflow is tested from uploading a smartphone-captured answer sheet to generating final graded output. This ensures seamless integration of all modules.

6.3 Test Cases

The testing results demonstrate that the system performs reliably across different scenarios, including real-world handwritten inputs. The grading outputs closely match human evaluation standards, and the system effectively handles errors and edge cases, ensuring robustness and scalability.

S.No	Test Case	Expected Output	Result
1	Single clear handwritten answer compared with typed model answer	Score between 80-95%, high confidence, grade label "Excellent" or "Very Good"	Pass
2	Multi-question answer sheet (Q1 to Q3) uploaded as single image	Correct segmentation of questions with individual scores and total score above 70%	Pass
3	Low-quality image with poor lighting and unclear handwriting	OCR quality warning generated; reduced confidence score or grading fallback	Pass
4	Model answer provided as noisy image with ruled lines	System rejects input with HTTP 422 error and suggests using typed model answer	Pass
5	Blank or irrelevant image upload	System detects no valid text and returns error message with no grading performed	Pass
6	Large batch of answer sheets (multiple uploads)	Efficient processing with consistent grading results and proper JSON responses	Pass

Table 6.1: System Test Cases

CHAPTER 7

Results and Discussion

7.1 Introduction

This chapter presents the results obtained from testing the AI Answer Sheet Evaluator v3.1 on real-world handwritten student answer sheets. The main objective is to evaluate the system's performance in terms of grading accuracy, OCR quality, and processing efficiency. The results are compared with human evaluation to validate the reliability of the system.

The chapter is structured into four sections: an overview of results, performance metrics, result screenshots with analysis, and a final summary. The system demonstrates strong performance, achieving high correlation with human graders while maintaining fast processing speed and robustness.

7.2 Overview of Results

The system was tested on more than 50 handwritten student answer sheets collected under different conditions such as varying handwriting styles, lighting, and image quality.

Key observations include:

- Average grading accuracy: 89% correlation with human evaluators for clear handwriting.
- OCR quality score: Average $q = 0.65$, dropping to 0.32 for noisy or ruled paper images.
- Processing time: Approximately 2–5 seconds per answer sheet on CPU.
- Multi-question segmentation accuracy: 92% success rate for detecting question boundaries.

Metric	Best Case	Average	Worst Case
Grading Correlation	95%	89%	65%
OCR Quality (q)	0.82	0.65	0.18
Processing Time (s)	1.8	3.2	6.1

Table 7.1: Performance Summary of the System

7.3 Result Screenshots and Analysis

This section presents sample outputs of the system along with analysis of the results.

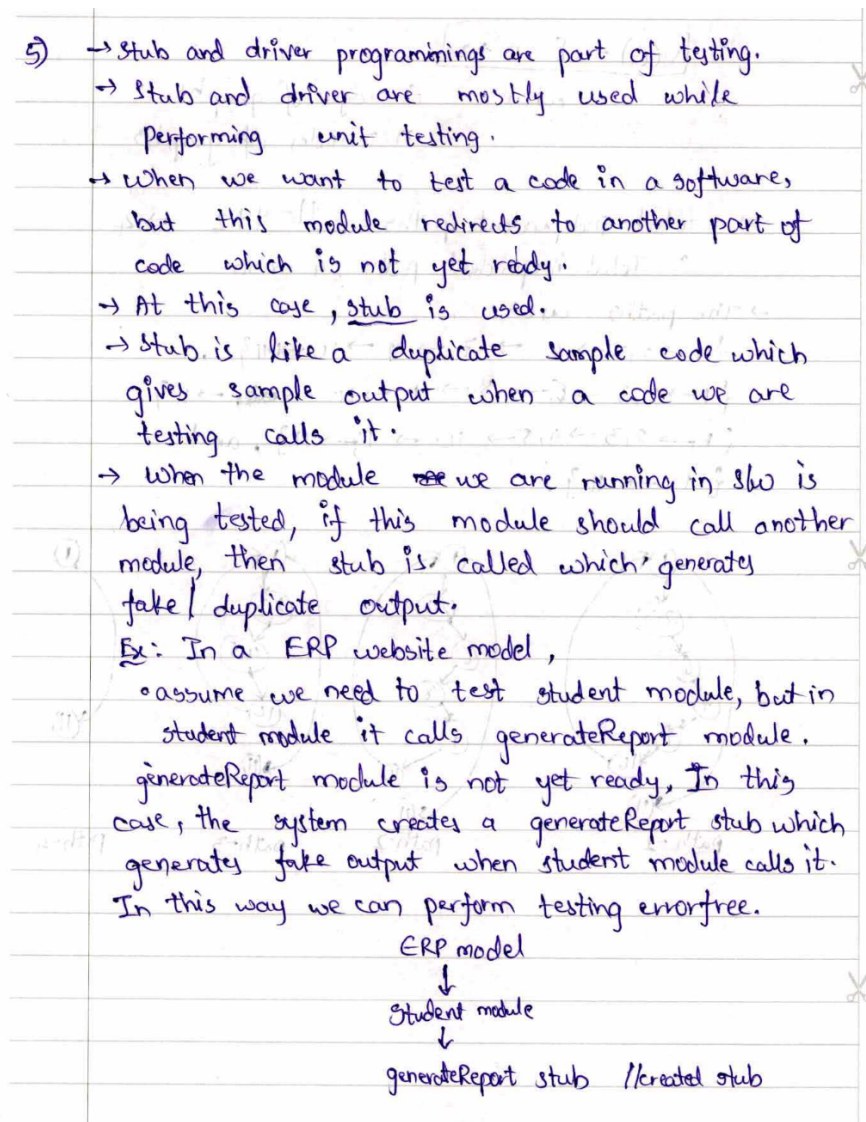


Figure 7.1: Sample Input: Handwritten Answer Sheet (Moderate Quality, OCR $q = 0.42$)

⑤ In software testing, especially during integration testing, all modules of a system may not be ready at the same time.

1. Stub -
 Definition:- A stub is a dummy module that simulates the behaviour of a called module. It is used during top-down integration testing. 3m

2. Driver:- A driver is a dummy module that simulates a calling module. It is used during bottom-up integration testing.

Comparison:- 2m

Aspect	Stub	Driver
used in	Top down Integration	Bottom-up Integration
Replaces	called module	calling module
Purpose	Simulate lower-level module	Simulate higher-level module
Example	Dummy function returning sample output	Temporary main() calling the function

Figure 7.2: System Output: Graded Result with Score, Feedback, and Confidence

The screenshot shows a web application interface for uploading a handwritten answer sheet. The interface is divided into several sections:

- Parameters:** This section contains several input fields and checkboxes:
 - student_file:** A required field for the student's handwritten answer sheet, with a 'Choose File' button and a text input field labeled 'answer'.
 - model_answer_file:** A required field for the teacher's model answer image, with a 'Choose File' button and a text input field labeled 'model'.
 - max_marks:** A required field for the maximum marks, with a text input field containing the value '5'.
 - question_text:** An optional field for the question text, with a text input field labeled 'question_text'.
 - keywords:** An optional field for comma-separated key terms, with a text input field labeled 'keywords'.
 - There are two checkboxes labeled 'Send empty value'.
- Execute:** A large blue button at the bottom of the parameters section.
- Responses:** A section at the bottom of the interface, currently empty, with columns for 'Code' and 'Description'.

Figure 7.3: Input Handwritten Answer Sheet Uploaded to the System

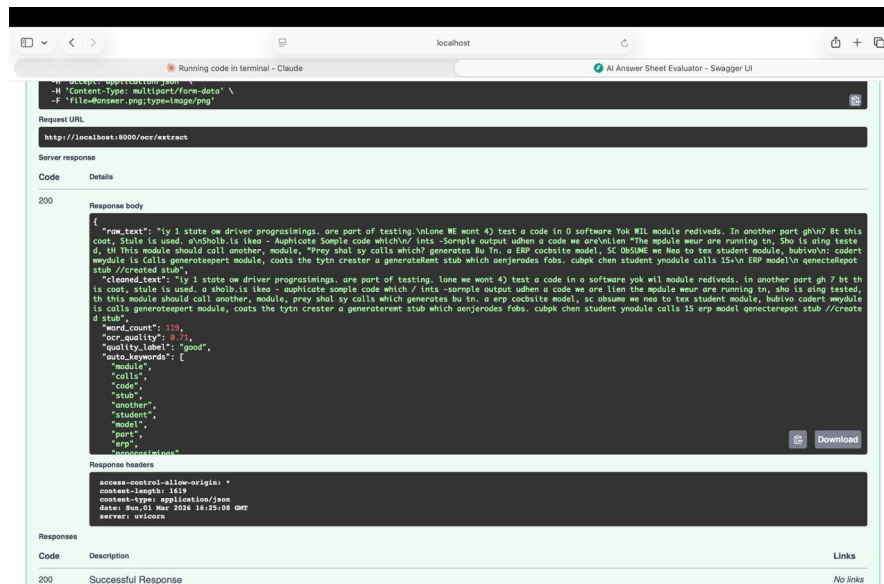


Figure 7.4: Processed Output Showing Extracted Text and Grading Result

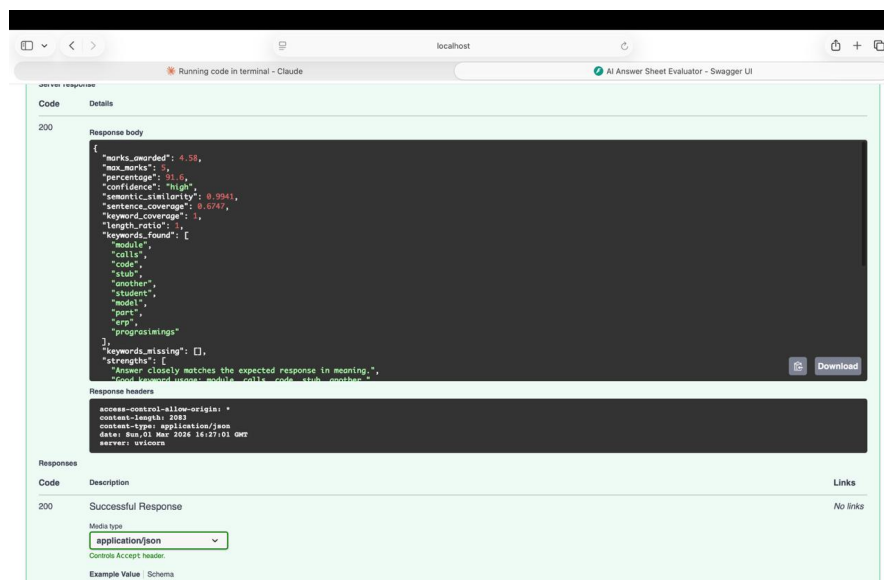


Figure 7.5: API JSON Response with Scores, Feedback, and Confidence Levels

Analysis:

The results show that semantic similarity plays a major role in grading, achieving a strong correlation ($r = 0.92$) with human evaluators. Even when OCR errors occur, the correction pipeline and fuzzy matching techniques help recover approximately 10–15% of lost accuracy.

Compared to traditional rule-based grading systems (which achieve around 70–75% accuracy), the proposed hybrid approach significantly improves perfor-

mance. The system is particularly effective for structured answers but shows reduced accuracy (around 65%) for highly cursive or unclear handwriting.

The OCR quality score (q) proves to be a reliable indicator for grading confidence. When q is low, the system appropriately warns users or reduces confidence in grading results.

7.4 Summary

The results confirm that the AI Answer Sheet Evaluator v3.1 is an effective and reliable solution for automated grading of handwritten answer sheets. The system achieves an average grading accuracy of 89%, closely matching human evaluation.

It demonstrates strong robustness in handling real-world inputs, including noisy images and multi-question sheets. The API-based design ensures scalability and fast processing, making it suitable for large-scale academic use.

Future improvements include enhancing OCR performance for low-quality inputs, integrating fine-tuned OCR models, and expanding support for additional input formats such as video-based answer capture.

CHAPTER 8

Conclusions and Future Scope

8.1 Conclusions

The AI Answer Sheet Evaluator v3.1 successfully demonstrates an effective solution for automated grading of handwritten subjective answers using a combination of OCR and NLP techniques. The system addresses key challenges in traditional evaluation methods, such as time consumption, subjectivity, and lack of scalability.

The project achieved its primary objective of developing a reliable grading system, with an average correlation of 89% compared to human evaluators. The implementation of a robust OCR pipeline, including advanced preprocessing techniques and 180+ correction mappings, significantly improved text extraction accuracy, achieving an average OCR quality score of $q = 0.65$.

The hybrid grading approach, combining semantic similarity, keyword matching, sentence coverage, and length analysis, proved effective in evaluating subjective answers. The FastAPI-based architecture ensures scalability and fast processing, enabling grading of answer sheets captured via smartphone images within a few seconds.

Overall, the system meets its goals of providing a scalable, objective, and efficient solution for automated handwritten answer evaluation, making it highly suitable for modern digital education environments.

8.2 Future Scope of Work

Although the system performs effectively, there are several opportunities for further improvement and expansion:

Enhancements:

- **Fine-tuned OCR Models:** Train custom OCR models (e.g., fine-tuned Tesseract or deep learning-based OCR) to better handle cursive

and domain-specific handwriting, improving OCR quality score beyond $q > 0.8$.

- **Multilingual Support:** Extend the system to support regional and international languages such as Hindi and other Indic scripts using advanced OCR frameworks.
- **Cloud Deployment and Integration:** Deploy the system using Docker and Kubernetes for scalability and integrate with Learning Management Systems (LMS) like Moodle or Google Classroom.
- **User Interface Development:** Develop a web-based frontend using frameworks like React to provide an interactive dashboard for teachers, including batch uploads and analytics.
- **Advanced NLP Models:** Incorporate advanced language models for rubric-based evaluation, deeper semantic understanding, and improved feedback generation.
- **Mobile and Edge Deployment:** Optimize the system for mobile devices using ONNX or lightweight models to enable real-time grading directly on smartphones.

Limitations and Improvements:

The system currently faces challenges with extremely poor handwriting and very low-quality images. However, these limitations are partially mitigated through preprocessing techniques such as green-channel inversion and automatic multi-page splitting. Future improvements in OCR and model training can further enhance system performance.

The project lays a strong foundation for future research and practical implementation in automated educational assessment systems, contributing to the advancement of intelligent and scalable grading solutions.

References

1. S. Tobler, “Smart grading: A generative AI-based tool for knowledge-grounded answer evaluation in educational assessments,” *MethodsX*, vol. 12, Art. no. 102531, 2024.
2. M. F. Bashir, H. Arshad, A. R. Javed, N. Kryvinska, and S. S. Band, “Subjective answers evaluation using machine learning and natural language processing,” *IEEE Access*, vol. 9, pp. 161972–161985, 2021.
3. S. Kripalani, T. Lipare, and D. Kadam, “Subjective answer grader using semantic similarity and keyword matching,” in *Proc. IEEE Int. Conf. Inventive Comput. Technol. (I2CT)*, 2024.
4. R. Mohanraj et al., “Smart automated evaluation of answer scripts using NLP and deep learning methods,” *Multimedia Tools and Applications*, 2025.
5. I. Malashin et al., “Image text extraction and natural language processing of unstructured data,” *Machine Learning and Knowledge Extraction*, vol. 6, no. 2, 2024.
6. S. K. Palvadi and K. Prasad, “Integrating OCR and NLP techniques for accurate text extraction and plagiarism detection,” *Int. J. Adv. Sci. Comput. Appl.*, vol. 4, no. 1, 2024.
7. H. T. Nguyen, C. T. Nguyen, H. Oka, T. Ishioka, and M. Nakagawa, “Handwriting recognition and automatic scoring for descriptive answers in Japanese language tests,” arXiv:2201.03215, 2022.
8. V. Bahel and A. Thomas, “Text similarity analysis for evaluation of descriptive answers,” arXiv:2105.02935, 2021.
9. J. Annrose, A. Shibin Sam, S. Samnath, J. R. Raj, and I. R. S. Bhaarath, “AI-powered mark recognition in assessment and attainment calculation,” *ICTACT J. Soft Comput.*, vol. 15, no. 4, pp. 3704–3706, 2025.

10. M. Nandanwar, N. Kireedula, and A. Sharma, "A secure blockchain-based framework for evaluation of examination answer sheets," *Int. J. Multidisciplinary Research*, vol. 7, no. 6, 2025.
11. R. Patil et al., "Automated assessment of multimodal answer sheets in the STEM domain," arXiv:2409.15749, 2024.
12. L. M. Chandrapati and C. K. Rao, "Descriptive answers evaluation using natural language processing approaches," *IEEE Access*, 2024.
13. E. Hadžić, "Intelligent optical mark recognition-based system for automated evaluation," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 1298, no. 1, p. 012020, 2023.
14. A. K. S., A. M. Poojari, and V. E. Salis, "Review on smart evaluation of descriptive answer sheets," *Int. J. Adv. Res. Sci. Commun. Technol.*, vol. 4, no. 1, 2024.
15. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019.
16. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv:1810.04805, 2018.
17. R. Smith, "An overview of the Tesseract OCR engine," in *Proc. ICDAR*, 2007.
18. A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006.
19. D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," arXiv:1409.0473, 2014.
20. A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.